

Chapitre 4 : Interroger les données en SQL

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Base de données 1, fondements, modélisation et SQL · **Fil rouge** : la clinique Santé Plus · **Version** : 2026-10-31

Fiche du chapitre

Objectifs (OS4)	A la fin du chapitre, l'apprenant sait : 1) écrire des requêtes de sélection avec conditions, tri et limite ; 2) utiliser les fonctions sur une ligne (textes, nombres, dates, `CASE`, `COALESCE`) ; 3) calculer des agrégats et regrouper (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, `GROUP BY`, `HAVING`) ; 4) combiner plusieurs tables par des jointures internes, externes et des auto-jointures ; 5) écrire des sous-requêtes (`IN`, `EXISTS`, sous-requêtes corrélées) ; 6) utiliser les opérateurs ensemblistes (`UNION`, `INTERSECT`, `EXCEPT`) ; 7) créer des vues ; 8) créer un index et lire un plan d'exécution simple.
Prérequis	Chapitres 1 à 3 ; atelier 3 terminé
Durée	🟢 Débutant : 14 h · 🔵 Standard : 10 h · 🟣 Expert : 7 h (atelier compris)
Matériel	Votre SGBD et le fichier fourni `clinique-complete.sql`
Livrable	L'atelier 4 (les requêtes de la clinique), puis le mini-projet de synthèse

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas. A sauter si le public est à l'aise.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Avant de commencer, exécutez le fichier fourni `clinique-complete.sql` dans votre base `clinique`. Il recrée les dix tables de l'atelier 3 avec leurs données de référence, et ajoute une patiente qui vient de s'inscrire (Bineta Cissé, n°9, aucune consultation) et un médicament jamais prescrit (Ibuprofène). Ainsi, **tout le monde a exactement les mêmes données**, et vos résultats doivent être identiques à ceux du chapitre.

Tous les résultats cités ont été obtenus à l'identique sur MySQL (MariaDB) et PostgreSQL. Ils sont affichés au format de PostgreSQL ; MySQL montre les mêmes valeurs dans des cadres `+---+`.

---

1. La requête SELECT

1.1 Choisir les colonnes

```
SELECT nom_patient, prenom_patient, ville
FROM patient
ORDER BY nom_patient, prenom_patient;
```

```
 nom_patient | prenom_patient |    ville
-------------+----------------+-------------
 Cissé       | Bineta         | Saint-Louis
 Diallo      | Ousmane        | Pikine
 Diop        | Awa            | Dakar
 Fall        | Ibrahima       | Rufisque
 Ndiaye      | Aliou          | Dakar
 Ndiaye      | Moussa         | Thiès
 Sarr        | Fatou          | Dakar
 Sy          | Mariama        | Dakar
 Thiam       | Khady          | Thiès
(9 lignes)
```

Morceau	Ce qu'il veut dire
`SELECT nom_patient, prenom_patient, ville`	Les colonnes à afficher, dans cet ordre. `SELECT *` les afficherait toutes
`FROM patient`	La table où chercher
`ORDER BY nom_patient, prenom_patient`	Trie par nom, puis, à nom égal (les deux Ndiaye), par prénom

L'ordre d'écriture d'une requête est toujours le même ; les clauses facultatives peuvent manquer, mais pas changer de place :

```
SELECT ...      les colonnes (et calculs) à afficher
FROM ...        la ou les tables
WHERE ...       les lignes à garder
GROUP BY ...    les regroupements           (section 4)
HAVING ...      les groupes à garder        (section 4)
ORDER BY ...    le tri
LIMIT ...       le nombre de lignes         (section 1.4)
```

1.2 Éliminer les doublons, renommer, calculer

```
SELECT DISTINCT ville FROM patient ORDER BY ville;

SELECT num_consultation, tarif, tarif * 0.8 AS part_assurance_80
FROM consultation
WHERE num_consultation <= 3
ORDER BY num_consultation;
```

```
    ville
-------------
 Dakar
 Pikine
 Rufisque
 Saint-Louis
 Thiès
(5 lignes)

 num_consultation | tarif | part_assurance_80
------------------+-------+------------------
                1 | 10000 |            8000.0
                2 | 15000 |           12000.0
                3 | 20000 |           16000.0
(3 lignes)
```

Morceau	Ce qu'il veut dire
`DISTINCT`	Supprime les lignes en double du résultat : chaque ville une seule fois (4 patients habitent Dakar)
`tarif * 0.8`	Une **colonne calculée** : elle n'existe pas dans la table, elle est calculée pour l'affichage. On peut utiliser `+`, `-`, `*`, `/`
`AS part_assurance_80`	Un **alias** : le nom affiché en en-tête de la colonne

1.3 Filtrer avec `WHERE`

Opérateur	Sens	Exemple
`=`, `<>`, `<`, `<=`, `>`, `>=`	Comparaison (`<>` : différent)	`tarif >= 15000`
`AND`, `OR`, `NOT`	Combiner des conditions	`ville = 'Dakar' AND sexe = 'F'`
`BETWEEN a AND b`	Entre a et b, **bornes comprises**	`tarif BETWEEN 12000 AND 20000`
`IN (...)`	Égal à l'une des valeurs	`motif IN ('Fièvre', 'Paludisme')`
`LIKE`	Ressemble à un modèle : `%` = n'importe quelle suite de caractères, `_` = un seul caractère	`nom_patient LIKE 'D%'`
`IS NULL`, `IS NOT NULL`	Absence (ou présence) de valeur	`code_assurance IS NULL`

```sql
SELECT nom_patient, prenom_patient FROM patient WHERE nom_patient LIKE 'D%' ORDER BY
        prenom_patient;
SELECT nom_patient, prenom_patient FROM patient WHERE code_assurance IS NULL ORDER BY num_patient;
```

Résultats : `Diop Awa`, `Diallo Ousmane` ; puis les cinq patients sans assurance : Sarr Fatou, Fall Ibrahima, Diallo Ousmane, Ndiaye Aliou, Cissé Bineta.

> 🖍️ **Essayez : la priorité de `AND` sur `OR`.** Comparez ces deux requêtes :
>
> ```
> SELECT num_consultation FROM consultation
> WHERE motif = 'Fièvre' OR motif = 'Paludisme' AND tarif > 10000;
>
> SELECT num_consultation FROM consultation
> WHERE (motif = 'Fièvre' OR motif = 'Paludisme') AND tarif > 10000;
> ```
>
> La première donne les consultations **1 et 5**, la seconde seulement la **5**. Comme la multiplication avant l'addition, `AND` est calculé **avant** `OR` : la première se lit « Fièvre (à n'importe quel prix), **ou** (Paludisme à plus de 10000) ». Dès que vous mélangez `AND` et `OR`, **mettez des parenthèses**.

1.4 Trier et limiter

```
SELECT num_consultation, date_consultation, tarif
FROM consultation
ORDER BY tarif DESC, date_consultation
LIMIT 3;
```

```
 num_consultation | date_consultation | tarif
------------------+-------------------+-------
                3 | 2026-10-12        | 20000
                6 | 2026-10-19        | 20000
                2 | 2026-10-05        | 15000
(3 lignes)
```

Morceau	Ce qu'il veut dire
`ORDER BY tarif DESC`	Du plus cher au moins cher (`DESC` : décroissant ; `ASC`, croissant, est la valeur par défaut)
`, date_consultation`	A tarif égal, la plus ancienne d'abord
`LIMIT 3`	Les trois premières lignes seulement

SGBD	Limiter à 3 lignes
MySQL	`LIMIT 3`
PostgreSQL	`LIMIT 3` ou, selon la norme, `FETCH FIRST 3 ROWS ONLY`
🔶 Oracle (12c et plus)	`FETCH FIRST 3 ROWS ONLY`

MySQL ne connaît pas `FETCH FIRST` : `ERROR 1064 ... near 'FETCH FIRST 3 ROWS ONLY'`.

2. Les fonctions sur une ligne

Une fonction transforme une valeur, **ligne par ligne**.

2.1 Textes, nombres, dates

```
SELECT UPPER(nom_patient) AS nom, LENGTH(prenom_patient) AS lettres, SUBSTRING(tel_patient, 1, 2)
         AS operateur
FROM patient WHERE num_patient <= 3 ORDER BY num_patient;
```

```
  nom   | lettres | operateur
--------+---------+-----------
 DIOP   |       3 | 77
 NDIAYE |       6 | 70
 SARR   |       5 | 76
(3 lignes)
```

Fonction	Rôle	🔶 Oracle
`UPPER(t)`, `LOWER(t)`	Majuscules, minuscules	Identique
`LENGTH(t)`	Longueur (voir l'encadré ci-dessous)	Identique
`SUBSTRING(t, debut, longueur)`	Extrait une partie : ici les 2 premiers chiffres du téléphone	`SUBSTR(t, debut, longueur)`
`CONCAT(a, ' ', b)`	Colle des textes (MySQL et PostgreSQL)	`a || ' ' || b` (`CONCAT` n'y prend que 2 textes)
`TRIM(t)`	Enlève les espaces au début et à la fin	Identique
`ROUND(n, d)`	Arrondit à `d` décimales	Identique
`CURRENT_DATE`	La date du jour	Identique
`EXTRACT(YEAR FROM d)`	L'année d'une date (`MONTH`, `DAY` aussi)	Identique
`CAST(x AS type)`	Convertit une valeur dans un autre type : `CAST('15000' AS INT)`	Identique

🔵 **Standard : `LENGTH` et les accents** `SELECT ville, LENGTH(ville), CHAR_LENGTH(ville) FROM patient WHERE num_patient = 2;` donne pour `Thiès` : **6** et 5 sur MySQL, mais 5 et 5 sur PostgreSQL. Sur MySQL, `LENGTH` compte les **octets**, et `è` en occupe deux ; pour compter les **caractères**, utilisez `CHAR_LENGTH`, qui donne 5 partout.

🔵 **Standard : `||` n'est pas le même partout** `SELECT prenom_patient || ' ' || nom_patient FROM patient WHERE num_patient = 1;` affiche `Awa Diop` sur PostgreSQL et Oracle, mais `0` sur MySQL, pour qui `||` veut dire « OU logique ». Sur MySQL, utilisez `CONCAT`.

2.2 Le nombre de jours entre deux dates

SGBD	Nuits d'un séjour
MySQL	`DATEDIFF(date_sortie, date_entree)`
PostgreSQL	`date_sortie - date_entree`
🔶 Oracle	`date_sortie - date_entree`

> ⚠️ **Le piège de MySQL.** Sur MySQL, `date_sortie - date_entree` ne provoque **pas** d'erreur : MySQL transforme les dates en nombres (`20261102` et `20261028`) et les soustrait. Pour du 28 octobre au 2 novembre 2026, il répond **74** au lieu de 5. Le résultat est juste par hasard quand les deux dates sont dans le même mois. Sur MySQL, utilisez toujours `DATEDIFF` .

2.3 `COALESCE` et `CASE`

```sql
SELECT nom_patient, COALESCE(code_assurance, 'aucune') AS assurance
FROM patient WHERE num_patient <= 4 ORDER BY num_patient;

SELECT num_consultation, tarif,
       CASE WHEN tarif >= 20000 THEN 'spécialiste'
            WHEN tarif >= 15000 THEN 'standard'
            ELSE 'générale'
       END AS categorie
FROM consultation WHERE num_consultation <= 4 ORDER BY num_consultation;
```

```
 nom_patient | assurance
-------------+-----------
 Diop        | IPM-SONA
 Ndiaye      | MUT-ENS
 Sarr        | aucune
 Fall        | aucune
(4 lignes)

 num_consultation | tarif |  categorie
------------------+-------+-------------
                1 | 10000 | générale
                2 | 15000 | standard
                3 | 20000 | spécialiste
                4 | 15000 | standard
(4 lignes)
```

Morceau	Ce qu'il veut dire
`COALESCE(code_assurance, 'aucune')`	Renvoie la **première valeur non** `NULL` de la liste : l'assurance si elle existe, sinon `'aucune'`

Morceau	Ce qu'il veut dire
`CASE WHEN ... THEN ... ELSE ... END`	Un « si... alors... sinon » dans une requête. Les conditions sont testées **dans l'ordre** : la première vraie l'emporte (un tarif de 20000 est aussi `>= 15000`, mais il est classé « spécialiste »)

3. Les agrégats et les regroupements

3.1 Les fonctions d'agrégat

Une fonction d'**agrégat** résume **plusieurs lignes** en une seule valeur.

```
SELECT COUNT(*) AS nb, SUM(tarif) AS total, MIN(tarif) AS mini, MAX(tarif) AS maxi
FROM consultation;

SELECT COUNT(*) AS patients, COUNT(code_assurance) AS assures, COUNT(DISTINCT ville) AS villes
FROM patient;
```

```
 nb | total  | mini  | maxi
----+--------+-------+-------
 10 | 140000 | 10000 | 20000
(1 ligne)

 patients | assures | villes
----------+---------+--------
        9 |       4 |      5
(1 ligne)
```

Fonction	Ce qu'elle calcule
`COUNT(*)`	Le nombre de **lignes**
`COUNT(colonne)`	Le nombre de valeurs **non** `NULL` de la colonne : 4 patients ont une assurance
`COUNT(DISTINCT colonne)`	Le nombre de valeurs **différentes**
`SUM`, `AVG`	La somme, la moyenne (`NULL` ignorés)
`MIN`, `MAX`	La plus petite, la plus grande valeur

`SELECT ROUND(AVG(tarif), 0) FROM consultation;` donne `14000` : 140000 / 10.

3.2 `GROUP BY` : un résultat par groupe

```
SELECT num_medecin, COUNT(*) AS nb, SUM(tarif) AS chiffre
FROM consultation
GROUP BY num_medecin
ORDER BY num_medecin;
```

```
 num_medecin | nb | chiffre
-------------+----+---------
           1 |  3 |   30000
           2 |  2 |   30000
           3 |  2 |   40000
           4 |  1 |   15000
           5 |  1 |   15000
           6 |  1 |   10000
(6 lignes)
```

Le SGBD range les consultations en **paquets**, un par médecin, puis calcule `COUNT` et `SUM` **dans chaque paquet**. Le médecin n°1 a fait les consultations 1, 8 et 10 : 3 consultations, 10000 + 10000 + 10000 = 30000.

> 🟢 **Débutant : la règle d'or du `GROUP BY`** Dans le `SELECT`, on ne peut mettre que : les colonnes du `GROUP BY`, et des fonctions d'agrégat. Ajoutez `motif` à la requête ci-dessus (`SELECT num_medecin, motif, COUNT(*) ...`) : PostgreSQL refuse, avec `ERREUR: la colonne « consultation.motif » doit apparaître dans la clause GROUP BY ou doit être utilisé dans une fonction d'agrégat`. Le médecin n°1 a trois motifs : lequel afficher ? MariaDB, lui, accepte et affiche **un motif au hasard** du paquet : un résultat faux sans avertissement (MySQL 8 refuse, comme PostgreSQL, avec son réglage par défaut).

3.3 `HAVING` : filtrer les groupes

```sql
SELECT num_medecin, COUNT(*) AS nb
FROM consultation
GROUP BY num_medecin
HAVING COUNT(*) >= 2
ORDER BY num_medecin;
```

Résultat : les médecins 1 (3 consultations), 2 et 3 (2 chacun).

	`WHERE`	`HAVING`
Filtre	les **lignes**, avant le regroupement	les **groupes**, après le regroupement
Peut contenir un agrégat ?	**Non**	Oui

```sql
SELECT ville, COUNT(*) AS nb
FROM patient
WHERE sexe = 'F'
GROUP BY ville
HAVING COUNT(*) >= 2;
```

Résultat : `Dakar | 3`. Le `WHERE` garde d'abord les patientes, le `GROUP BY` les range par ville, le `HAVING` garde les villes qui en ont au moins deux.

> 🧨 **Essayez** : `SELECT num_medecin FROM consultation WHERE COUNT(*) >= 2 GROUP BY num_medecin;`
>
> - MySQL : `ERROR 1111 (HY000): Invalid use of group function`
> - PostgreSQL : `ERREUR: les fonctions d'agrégat ne sont pas autorisées dans WHERE`
>
> Au moment du `WHERE`, les groupes n'existent pas encore : la condition sur `COUNT(*)` va dans `HAVING`.

4. Les jointures : interroger plusieurs tables

4.1 La jointure interne

La consultation ne contient que le **numéro** du patient. Pour afficher son **nom**, on **joint** les deux tables sur leur colonne commune :

```sql
SELECT c.num_consultation, c.date_consultation, p.nom_patient, p.prenom_patient
FROM consultation c
JOIN patient p ON p.num_patient = c.num_patient
WHERE c.num_consultation <= 4
ORDER BY c.num_consultation;
```

```
 num_consultation | date_consultation | nom_patient | prenom_patient
------------------+-------------------+-------------+----------------
                1 | 2026-10-05        | Diop        | Awa
                2 | 2026-10-05        | Sarr        | Fatou
                3 | 2026-10-12        | Ndiaye      | Moussa
                4 | 2026-10-14        | Sy          | Mariama
(4 lignes)
```

Morceau	Ce qu'il veut dire
`FROM consultation c`	La table de départ, avec un **alias** court `c`
`JOIN patient p`	On y ajoute la table `patient`, alias `p` (on peut écrire `INNER JOIN`)
`ON p.num_patient = c.num_patient`	La **condition de jointure** : chaque consultation est collée au patient qui a **le même numéro**. C'est la clé étrangère qui rejoint la clé primaire
`c.date_consultation`, `p.nom_patient`	`alias.colonne` : on précise de quelle table vient chaque colonne (obligatoire quand deux tables ont une colonne de même nom, comme `num_patient`)

On peut enchaîner autant de jointures que nécessaire, en suivant les clés étrangères du MLD :

```sql
SELECT c.num_consultation, p.nom_patient, m.nom_medecin, s.nom_service
FROM consultation c
JOIN patient p ON p.num_patient = c.num_patient
JOIN medecin m ON m.num_medecin = c.num_medecin
JOIN service s ON s.code_service = m.code_service
WHERE c.date_consultation >= DATE '2026-10-19'
ORDER BY c.num_consultation;
```

```
 num_consultation | nom_patient | nom_medecin |       nom_service
------------------+-------------+-------------+-------------------------
                6 | Thiam       | Sow         | Cardiologie
                7 | Fall        | Gueye       | Consultations générales
                8 | Diop        | Ba          | Consultations générales
                9 | Sarr        | Faye        | Pédiatrie
               10 | Ndiaye      | Ba          | Consultations générales
(5 lignes)
```

> 🧨 **Essayez : la jointure oubliée.** `SELECT COUNT(*) FROM consultation, patient;` (deux tables, sans condition) donne **90** : chaque consultation (10) est associée à **chaque** patient (9). C'est le **produit cartésien**, presque toujours une erreur : il manque la condition de jointure.

4.2 La jointure externe : garder les lignes sans correspondance

Une jointure interne ne garde que les lignes qui **ont** une correspondance. Bineta Cissé n'a aucune consultation : elle disparaît. Pour la garder, on utilise une **jointure externe gauche** :

```sql
SELECT p.num_patient, p.nom_patient, COUNT(c.num_consultation) AS nb
FROM patient p
LEFT JOIN consultation c ON c.num_patient = p.num_patient
GROUP BY p.num_patient, p.nom_patient
ORDER BY p.num_patient;
```

```
 num_patient | nom_patient | nb
-------------+-------------+----
           1 | Diop        |  2
           2 | Ndiaye      |  1
           3 | Sarr        |  2
           4 | Fall        |  1
           5 | Sy          |  1
           6 | Diallo      |  1
           7 | Thiam       |  1
           8 | Ndiaye      |  1
           9 | Cissé       |  0
(9 lignes)
```

Morceau	Ce qu'il veut dire
`LEFT JOIN`	Garde **toutes** les lignes de la table de **gauche** (`patient`), même sans consultation. Pour elles, les colonnes de `consultation` valent `NULL`
`COUNT(c.num_consultation)`	Compte les valeurs non `NULL` : 0 pour Bineta. Avec `COUNT(*)` , on aurait compté 1 (sa ligne existe dans le résultat)
`RIGHT JOIN`	L'inverse : garde toutes les lignes de la table de droite. On l'évite en mettant la table à garder à gauche

Le même principe trouve ce qui **n'a pas** de correspondance :

```
SELECT m.code_medicament, m.nom_medicament
FROM medicament m
LEFT JOIN prescription pr ON pr.code_medicament = m.code_medicament
WHERE pr.code_medicament IS NULL;
```

Résultat : `IBU400 | Ibuprofène 400 mg` , le seul médicament jamais prescrit.

Et voici le **reste à payer** de chaque consultation (le tarif moins la somme de ses paiements) :

```
SELECT c.num_consultation, c.tarif,
       COALESCE(SUM(pa.montant), 0) AS paye,
       c.tarif - COALESCE(SUM(pa.montant), 0) AS reste
FROM consultation c
LEFT JOIN paiement pa ON pa.num_consultation = c.num_consultation
GROUP BY c.num_consultation, c.tarif
HAVING c.tarif - COALESCE(SUM(pa.montant), 0) > 0
ORDER BY c.num_consultation;
```

```
 num_consultation | tarif | paye  | reste
------------------+-------+-------+-------
                4 | 15000 | 12000 |  3000
                7 | 10000 |  5000 |  5000
                8 | 10000 |  8000 |  2000
                9 | 15000 |     0 | 15000
               10 | 10000 |     0 | 10000
(5 lignes)
```

`COALESCE(SUM(...), 0)` est indispensable : pour une consultation sans paiement, `SUM` vaut `NULL` , et `10000 - NULL` vaut `NULL` , pas 10000.

4.3 L'auto-jointure

Une table peut être jointe **à elle-même**, avec deux alias. Exemple : les paires de patients qui habitent la même ville (pour organiser un transport groupé) :

```sql
SELECT p1.prenom_patient AS patient1, p2.prenom_patient AS patient2, p1.ville
FROM patient p1
JOIN patient p2 ON p1.ville = p2.ville AND p1.num_patient < p2.num_patient
ORDER BY p1.ville, p1.num_patient, p2.num_patient;
```

Résultat : 7 paires (6 à Dakar, entre Awa, Fatou, Mariama et Aliou ; 1 à Thiès, Moussa et Khady). La condition `p1.num_patient < p2.num_patient` évite d'associer un patient à lui-même, et d'avoir chaque paire deux fois (Awa-Fatou et Fatou-Awa).

5. Les sous-requêtes

Une **sous-requête** est une requête écrite **à l'intérieur** d'une autre, entre parenthèses.

5.1 Dans le `WHERE`

```sql
SELECT num_consultation, tarif FROM consultation
WHERE tarif > (SELECT AVG(tarif) FROM consultation)
ORDER BY num_consultation;
```

Résultat : les consultations 2, 3, 4, 5, 6 et 9, plus chères que la moyenne (14000). La sous-requête calcule d'abord **une seule valeur** (14000), puis la requête principale l'utilise.

```sql
SELECT nom_patient, prenom_patient FROM patient
WHERE num_patient IN (SELECT num_patient FROM sejour)
ORDER BY num_patient;
```

Résultat : Sy Mariama, Diallo Ousmane, Thiam Khady (les patients hospitalisés au moins une fois). La sous-requête renvoie **une liste** de numéros, utilisée par `IN`.

5.2 `NOT IN` et `NOT EXISTS`

```sql
SELECT nom_patient, prenom_patient FROM patient
WHERE num_patient NOT IN (SELECT num_patient FROM consultation);

SELECT m.nom_medicament FROM medicament m
WHERE NOT EXISTS (SELECT 1 FROM prescription pr WHERE pr.code_medicament = m.code_medicament);
```

Résultats : Cissé Bineta ; Ibuprofène 400 mg. `EXISTS (...)` est vrai si la sous-requête renvoie **au moins une ligne** ; `NOT EXISTS`, si elle n'en renvoie aucune. Ce qu'elle sélectionne (ici `1`) n'a pas d'importance.

> ⚠️ **Le piège de `NOT IN` et de `NULL`.** « Les patients dont l'assurance n'est pas une assurance de patient dakarois » :
>
> ```
> SELECT COUNT(*) FROM patient
> WHERE code_assurance NOT IN (SELECT code_assurance FROM patient WHERE ville = 'Dakar');
> ```
>
> On attend 2 (Moussa, mutuelle des enseignants ; Khady, assurance privée). Le résultat est **0**, sur tous les SGBD. Des Dakarois sans assurance mettent un `NULL` dans la liste, et « différent de `NULL` » n'est jamais vrai. Ajoutez `WHERE ... AND code_assurance IS NOT NULL` dans la sous-requête, ou utilisez `NOT EXISTS`, qui n'a pas ce problème.

5.3 La sous-requête corrélée

Une sous-requête **corrélée** utilise une colonne de la requête principale : elle est recalculée **pour chaque ligne**. La dernière consultation de chaque patient :

```
SELECT c.num_patient, c.num_consultation, c.date_consultation
FROM consultation c
WHERE c.date_consultation = (SELECT MAX(c2.date_consultation)
                             FROM consultation c2
                             WHERE c2.num_patient = c.num_patient)
ORDER BY c.num_patient;
```

```
 num_patient | num_consultation | date_consultation
-------------+------------------+-------------------
           1 |                8 | 2026-10-21
           2 |                3 | 2026-10-12
           3 |                9 | 2026-10-22
           4 |                7 | 2026-10-20
           5 |                4 | 2026-10-14
           6 |                5 | 2026-10-15
           7 |                6 | 2026-10-19
           8 |               10 | 2026-10-26
(8 lignes)
```

Pour la consultation n°1 (patient 1), la sous-requête cherche la date maximale **des consultations du patient 1** : le 21 octobre. La consultation 1 (5 octobre) n'est pas gardée, la 8 l'est.

5.4 Dans le `FROM`

Une sous-requête peut servir de **table temporaire**, avec un alias :

```
SELECT ROUND(AVG(nb), 2) AS moyenne_consultations_par_medecin
FROM (SELECT num_medecin, COUNT(*) AS nb FROM consultation GROUP BY num_medecin) t;
```

Résultat : `1.67` (10 consultations pour 6 médecins).

6. Les opérateurs ensemblistes

Ils combinent les résultats de **deux requêtes** qui ont le **même nombre de colonnes**, de types compatibles.

Opérateur	Résultat	🔶 Oracle
`UNION`	Les lignes de l'une **ou** de l'autre, sans doublon	Identique
`UNION ALL`	Toutes les lignes des deux, doublons compris	Identique
`INTERSECT`	Les lignes présentes dans **les deux**	Identique
`EXCEPT`	Les lignes de la première **qui ne sont pas** dans la seconde	`MINUS`

```
SELECT tel_patient AS telephone, 'patient' AS qui FROM patient WHERE ville = 'Thiès'
UNION
SELECT tel_medecin, 'médecin' FROM medecin WHERE code_service = 'PED'
ORDER BY telephone;
```

```
  telephone   |   qui
--------------+---------
 70 444 55 66 | patient
 76 300 40 50 | médecin
 76 444 55 66 | patient
 78 200 30 40 | médecin
(4 lignes)
```

Avec les villes des patientes et celles des patients :

Requête	Résultat
`... WHERE sexe = 'F' UNION ... WHERE sexe = 'M'`	Dakar, Pikine, Rufisque, Saint-Louis, Thiès
`UNION ALL` (en comptant les lignes)	9 lignes, une par patient
`INTERSECT`	Dakar, Thiès (des patients des deux sexes)
`EXCEPT` (femmes sauf hommes)	Saint-Louis

`INTERSECT` et `EXCEPT` existent sur MySQL depuis la version 8.0.31 (MariaDB depuis la 10.3).

7. Les vues

Une **vue** est une requête **enregistrée sous un nom**, qu'on interroge ensuite comme une table. Elle ne stocke pas de données : la requête est exécutée à chaque fois.

```sql
CREATE VIEW v_consultation_detail AS
SELECT c.num_consultation, c.date_consultation, p.nom_patient, p.prenom_patient,
       m.nom_medecin, c.motif, c.tarif
FROM consultation c
JOIN patient p ON p.num_patient = c.num_patient
JOIN medecin m ON m.num_medecin = c.num_medecin;

SELECT nom_patient, prenom_patient, nom_medecin, motif
FROM v_consultation_detail
WHERE motif = 'Paludisme'
ORDER BY num_consultation;
```

```
 nom_patient | prenom_patient | nom_medecin |   motif
-------------+----------------+-------------+-----------
 Fall        | Ibrahima       | Gueye       | Paludisme
 Ndiaye      | Aliou          | Ba          | Paludisme
(2 lignes)
```

A quoi sert une vue	Exemple
Simplifier : écrire une fois une jointure compliquée	La secrétaire interroge `v_consultation_detail` sans écrire de jointure
Sécuriser : ne montrer qu'une partie des données (le niveau externe du chapitre 1)	Une vue sans le motif, pour la caisse, avec des droits sur la vue et pas sur la table
Nommer un calcul métier	Une vue `v_impayes` (le reste à payer de la section 4.2) : `SELECT SUM(reste) FROM v_impayes;` donne `35000`

Supprimer une vue : `DROP VIEW v_consultation_detail;` (les tables ne sont pas touchées).

8. Les index et le plan d'exécution

8.1 L'index, comme celui d'un livre

Pour trouver un mot dans un livre, on peut lire toutes les pages... ou consulter l'**index** à la fin, qui donne directement la page. Un **index** de base de données fait la même chose pour une colonne : il permet de trouver les lignes qui ont une valeur donnée **sans lire toute la table**.

```sql
CREATE INDEX idx_consultation_date ON consultation (date_consultation);
CREATE INDEX idx_patient_nom ON patient (nom_patient, prenom_patient);
```

Bonne idée d'index	Mauvaise idée
Colonnes souvent dans `WHERE`, `JOIN`, `ORDER BY` (dates, noms recherchés, clés étrangères)	Colonnes rarement utilisées pour chercher
Grandes tables	Petites tables (quelques centaines de lignes : lire tout est aussi rapide)

Bonne idée d'index	Mauvaise idée
	Indexer toutes les colonnes : chaque index **ralentit** les `INSERT`, `UPDATE` et `DELETE`, qui doivent le mettre à jour

La clé primaire et les colonnes `UNIQUE` ont **automatiquement** un index. MySQL en crée aussi un pour chaque clé étrangère ; PostgreSQL et Oracle non : créez-les vous-même sur les grandes tables.

8.2 Lire un plan d'exécution

Le plan d'exécution montre **comment** le SGBD compte exécuter une requête. Pour le voir sur un vrai volume, l'archive des rendez-vous de la clinique compte 100 000 lignes (table `rdv_archive`, créée par le fichier fourni de l'atelier 4). On cherche les rendez-vous du patient 1234 :

```
EXPLAIN SELECT * FROM rdv_archive WHERE num_patient = 1234;
CREATE INDEX idx_rdv_patient ON rdv_archive (num_patient);
EXPLAIN SELECT * FROM rdv_archive WHERE num_patient = 1234;
```

	Avant l'index	Après l'index
PostgreSQL	`Seq Scan on rdv_archive` : lecture **séquentielle** de toute la table, puis `Filter: (num_patient = 1234)`	`Bitmap Index Scan on idx_rdv_patient` : passage par l'index, puis lecture des seules lignes trouvées
MySQL (colonnes de `EXPLAIN`)	`type: ALL`, `key: NULL`, `rows: 100083` (estimation) : toute la table	`type: ref`, `key: idx_rdv_patient`, `rows: 20`

Après l'index, le SGBD n'examine plus 100 000 lignes, mais **20** (les rendez-vous de ce patient). Le chiffre de MySQL avant l'index (100083) est une **estimation**, pas un compte exact.

> 🔶 **Oracle** `EXPLAIN PLAN FOR SELECT * FROM rdv_archive WHERE num_patient = 1234;` puis `SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);`. On y lit `TABLE ACCESS FULL` (toute la table) ou `INDEX RANGE SCAN` (par l'index). Dans FreeSQL et SQL Developer, un bouton affiche aussi le plan.

> 🟣 **Expert : pourquoi l'index n'est pas toujours utilisé** Sur nos petites tables de la clinique (10 consultations), PostgreSQL lit toute la table même avec un index : c'est plus rapide que de passer par l'index. L'**optimiseur** (chapitre 1, section 3.3) choisit selon des **statistiques** sur les données ; après de gros changements, on les met à jour avec `ANALYZE` (PostgreSQL) ou `ANALYZE TABLE` (MySQL).

Exercices

Sauf indication, les exercices portent sur la base de la clinique (fichier `clinique-complete.sql`). Écrivez la requête, puis vérifiez le résultat attendu.

- ★ **Exercice 1 : sélections.** a) Les patients de Thiès ou de Rufisque, triés par nom (3 lignes) ; b) les consultations dont le motif contient « Fièvre » ou « Paludisme » (4 lignes) ; c) les médecins dont le téléphone commence par `77` (2 lignes) ; d) les trois consultations les moins chères, les plus récentes d'abord à prix égal. (Section 1.)
- ★ **Exercice 2 : fonctions.** a) Le prénom et le nom en majuscules, collés, de chaque médecin (`MAMADOU BA` ...) ; b) l'année de naissance et le nom de chaque patiente ; c) pour chaque chambre, son numéro et « chère » si le prix par nuit dépasse 20000, « abordable » sinon ; d) pour chaque séjour terminé, le nombre de nuits (3 et 5). (Section 2.)
- ★★ **Exercice 3 : regroupements.** a) Le nombre de patients par ville, du plus grand au plus petit ; b) le montant total encaissé par jour de paiement ; c) les modes de paiement utilisés au moins 3 fois ; d) le nombre de prescriptions par médicament, y compris les médicaments jamais prescrits (avec 0). (Sections 3 et 4.2.)
- ★★ **Exercice 4 : jointures et sous-requêtes.** a) Pour chaque consultation de pédiatrie, le nom du patient, la date et le motif ; b) les patients venus **au moins deux fois** en consultation (2 patients) ; c) le médicament le plus prescrit (Paracétamol, 4 fois) ; d) les services dont **aucune chambre** n'a encore accueilli de séjour (1 service). (Sections 4 et 5.)
- ★★★ **Exercice 5 : la bibliothèque.** Sur votre base de l'exercice 5 du chapitre 3 (bibliothèque universitaire) : a) les emprunts non rendus avec le titre et le nom de l'étudiant ; b) le nombre d'emprunts par filière ; c) les livres jamais empruntés ; d) l'étudiant qui a le plus emprunté ; e) une vue `v_retards` des emprunts non rendus depuis plus de 15 jours (avec `CURRENT_DATE` et la fonction de dates de votre SGBD) ; f) l'index que vous créeriez si la table des emprunts avait un million de lignes, et pourquoi. (Toutes les sections.)

Atelier 4 et mini-projet

L'atelier 4 vous fait écrire les requêtes dont la clinique a besoin au quotidien, de la plus simple à la plus complète, avec une vue et un index. Le mini-projet de synthèse, en binôme, reprend toute la démarche du cours sur un sujet de votre choix. Les énoncés vous sont fournis séparément.

A retenir

- Ordre d'écriture : `SELECT` … `FROM` … `WHERE` … `GROUP BY` … `HAVING` … `ORDER BY` … `LIMIT`.
- `WHERE` : `=`, `<>`, `BETWEEN`, `IN`, `LIKE` (`%`, `_`), `IS NULL` ; `AND` passe avant `OR` : des parenthèses.
- Fonctions : `UPPER`, `LENGTH` (`CHAR_LENGTH` sur MySQL), `SUBSTRING`, `CONCAT`, `ROUND`, `EXTRACT`, `COALESCE`, `CASE`. Différences de dates : `DATEDIFF` sur MySQL, soustraction ailleurs.
- Agrégats : `COUNT(*)`, `COUNT(col)` (sans les `NULL`), `SUM`, `AVG`, `MIN`, `MAX`. Dans un `SELECT` groupé : seulement les colonnes du `GROUP BY` et des agrégats. `WHERE` filtre les lignes, `HAVING` les groupes.
- Jointure interne : `JOIN ... ON` clé étrangère = clé primaire ; externe : `LEFT JOIN` garde la table de gauche ; auto-jointure : deux alias de la même table. Oublier la condition : produit cartésien.
- Sous-requêtes : une valeur, une liste (`IN`), `EXISTS` ; corrélées (recalculées pour chaque ligne) ; dans le `FROM`. `NOT IN` + `NULL` = aucun résultat.
- `UNION`, `UNION ALL`, `INTERSECT`, `EXCEPT` (`MINUS` sur Oracle).

- Vue : une requête nommée. Index : accélère les recherches, ralentit les écritures ; `EXPLAIN` montre s'il est utilisé.

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`doit apparaître dans la clause GROUP BY ou doit être utilisé dans une fonction d'agrégat` (PostgreSQL)	Une colonne du `SELECT` n'est ni groupée ni agrégée	L'ajouter au `GROUP BY`, ou l'agréger
Un résultat « au hasard » sur MariaDB avec `GROUP BY`	Même erreur, que MariaDB accepte sans rien dire	Même solution
`Invalid use of group function` / `les fonctions d'agrégat ne sont pas autorisées dans WHERE`	Un agrégat dans le `WHERE`	Le mettre dans `HAVING`
`Column 'num_patient' in field list is ambiguous` / `la référence à la colonne « num_patient » est ambigüe`	Une colonne présente dans deux tables jointes, sans alias	Préfixer : `p.num_patient`
Beaucoup trop de lignes (90 au lieu de 10)	Condition de jointure oubliée : produit cartésien	Ajouter `JOIN ... ON`
Une ligne disparaît (un patient sans consultation)	Jointure interne au lieu d'externe	`LEFT JOIN`, et `COUNT(colonne)`
`NOT IN` ne renvoie rien	Un `NULL` dans la liste de la sous-requête	`IS NOT NULL` dans la sous-requête, ou `NOT EXISTS`
`10000 - NULL` donne `NULL`	Calcul avec une valeur absente	`COALESCE(valeur, 0)`
74 jours au lieu de 5 (MySQL)	Soustraction de deux dates	`DATEDIFF`
`ERROR 1064 ... near 'FETCH FIRST ...'` (MySQL)	Syntaxe de limite d'un autre SGBD	`LIMIT`

Pour aller plus loin

- MySQL, *SELECT* et *JOIN* : https://dev.mysql.com/doc/refman/8.4/en/select.html
- PostgreSQL en français, *Requêtes* : https://docs.postgresql.fr/16/queries.html ; *EXPLAIN* : https://docs.postgresql.fr/16/using-explain.html
- Oracle, *SQL Language Reference* (`SELECT`, fonctions) : https://docs.oracle.com/en/database/oracle/oracle-database/
- Oracle Academy, *Applied Database Systems*, section 2 (requêtes) ; schéma d'exemple HR sur FreeSQL pour s'entraîner.